Auditing KNIME Workflows in Scientific Studies: A Semi-Automated Evidence Pipeline

Vitalii Kaplan[0009-0009-8181-2863]

2333275007@alu.istinye.edu.tr

Abstract. In this paper, we study how visual workflows are reported in scientific publications and how incomplete workflow preservation can make older studies harder to reproduce. Using KNIME as a case study, we develop and apply a semi-automated evidence pipeline that combines OpenAlex bibliometrics, a 100-record top-cited article audit registry, workflow retrieval, and opening and execution checks in a current KNIME environment. The OpenAlex search shows that KNIME is visible across a broad scientific literature, while the article assessment shows that workflow figures and textual descriptions are more common than downloadable workflow artifacts. Of the 100 audited records, 22 report downloadable or linked KNIME workflows, and recorded retrieval attempts obtained workflow artifacts or workflow directories for 18. In the recorded manual checks, five article records had at least one workflow execute successfully. These results show that incomplete artifact and environment preservation limits the long-term reproducibility of KNIME-based studies.

Keywords: Workflow reproducibility · KNIME · Bibliometrics · Research artifacts

1 Introduction

KNIME is a widely used open-source platform for constructing scientific data-analysis workflows, with more than a decade of use across published studies [1]. Its visual workflows represent analyses as connected nodes with stored settings, data ports, and extension dependencies, which can make computational work easier to inspect, reuse, and repeat. This representation provides more operational detail than a narrative method description, but it does not remove the need to preserve the executable workflow artifact and its context. Reproduction requires access to the workflow file, KNIME version, extensions, data, and execution environment. Figures and textual summaries do not provide sufficient substitutes for these materials.

This issue matters for visual workflow platforms in general, because their reproducibility value depends on preserving executable workflow artifacts and the software context needed to interpret them. We study KNIME because it is an open-source visual workflow platform with a long public development history and substantial visibility in scientific literature. Our OpenAlex query returned 963 article-type records whose titles or abstracts match `KNIME`. The matching records span platform papers, extension papers, tool-comparison papers, and domain studies that use KNIME as an analysis interface. KNIME is therefore a suitable case for studying whether visual workflows are preserved as executable artifacts or only reported through descriptions and figures.

Our top-cited article collection covers the 100 most-cited KNIME-matching records, with not-assessed placeholders retained where local full text is unavailable. The structured audit reported in Table 2 shows an uneven publication pattern. Twenty-two records report downloadable or linked KNIME workflow files, and workflow artifacts or workflow directories were obtained for 18 article records in the recorded retrieval attempts. Other links were stale, blocked, incomplete, or did not lead to a confirmed KNIME workflow artifact. In the recorded manual execution checks, five article records had at least one workflow execute successfully. By comparison, 49 records present workflow screenshots or figures, and 68 describe workflow steps or nodes in text. These forms of presentation do not preserve the complete executable workflow context.

These reporting practices create a reproducibility risk because later readers may lack the executable artifact, input data, recorded KNIME version, or extension context needed to reconstruct and run the analysis. We therefore combine OpenAlex bibliometrics, top-cited article retrieval and assessment, workflow-artifact retrieval, and current-version opening and execution attempts. The goal is to measure how KNIME workflows are reported and preserved, and to distinguish publication-level claims of workflow availability from artifacts that were actually obtained and tested.

The rest of the paper is organized as follows. Section 2 reviews related work on computational reproducibility, open-source workflow systems, environment preservation, and research artifacts. Section 3 presents the bibliometric method and semi-automated evidence pipeline. Section 4 describes the resulting datasets. Section 5 reports the bibliometric, article-audit, workflow-retrieval, and manual opening and execution results. Section 6 discusses the implications, limitations, and future extensions, and Section 7 concludes the paper.

2 Literature Review

Reproducibility is a central requirement for computational research because published results increasingly depend on software, data transformations, and execution environments. Provenance research makes this requirement explicit: visual analytics systems need records of data, analytical processes, and human interpretation to make later inspection and reuse possible [19]. Studies of executable research artifacts show the same problem in practice. Samuel and Mietchen found that biomedical Jupyter notebooks often fail during dependency installation or re-execution even when the notebooks themselves are available [15].

Open-source scientific software helps reproducibility by making algorithms, interfaces, and execution infrastructure inspectable. Weka, for example, is presented as an extensible open-source data-mining environment with graphical interfaces, reusable configurations, and plugin mechanisms [5]. KNIME is similarly used as an open-source visual workflow platform, and recent GIScience work explicitly proposes KNIME-based workflows as an approach for advancing replicable and reproducible research [3]. However, openness of the platform alone is insufficient. A workflow-based study also requires the executable workflow file, input data, platform version, installed extensions, and other software dependencies. Container research makes the same point at the environment level: scientific computations are easier to reproduce when complete software environments can be preserved and moved between systems [9]. More broadly, the testing literature shows that executable artifacts can remain sensitive to operating systems, dependency versions, timing, external services, and other environmental factors [17].

For visual workflow studies, preservation must include more than an image of the workflow graph. Node settings, extension dependencies, input paths, and version context are part of the executable method. The distinction between a reported workflow, a downloadable artifact, an artifact that opens, and one that executes successfully is therefore central to the present audit.

3 Methods

All scripts and data used for this study, except the other articles assessed as study objects, are available in the public GitHub repository [8]. Relative paths reported in this paper should be interpreted as paths within that repository.

3.1 Bibliometric Evidence

We collected bibliometric evidence from the OpenAlex Works API [13] on June 29, 2026. The query used OpenAlex title-and-abstract search for the word `KNIME` and restricted the result set to OpenAlex records whose type is `article`. The same query can be reproduced in the OpenAlex web interface by selecting "Search title and abstract", entering `KNIME`, and filtering the result type to `article`. The corresponding web-interface request can be represented as:

```
https://openalex.org/works
page=1
sort=relevance_score:desc
search.title_and_abstract=KNIME
filter=type:article
```

The first API request, which returns the first 200 records, is reproducible as:

```
https://api.openalex.org/works
?search.title_and_abstract=KNIME
&filter=type:article
&per-page=200
&cursor=*
```

OpenAlex uses cursor pagination for large result sets. After the first request, the collector reads `meta.next_cursor` from each response and submits the same request with `cursor=<next-cursor>` until no further results are returned. The value `per-page=200` is the maximum page size used here.

The collection script is `scripts/collect_openalex_knime_works.py`. It stores raw and processed collection artifacts as follows:

- raw OpenAlex response pages: `data/original/openalex/pages/`
- complete record stream: `data/original/openalex/works.jsonl`
- compact CSV view: `data/original/openalex/works.csv`
- endpoint, query parameters, collection time, result count, and page manifest: `data/original/openalex/metadata.json`

The current run collected 963 records across 5 OpenAlex response pages. Derived bibliometric tables are generated from the JSONL file by `scripts/build_openalex_knime_bibliometrics.py`.

3.2 Semi-Automated Evidence Pipeline

For the article-level audit, we developed an open-source, semi-automated evidence pipeline composed of scripts and JSON configuration files. The process is specified in `scripts/audit_script_chain.json`, which serves as the workflow index for this part of the study. Each step consumes recorded outputs from earlier steps and writes an explicit intermediate or final artifact. The current chain contains fifteen scripted steps and one manual checkpoint. Thirteen scripted steps are rule-based. Two scripted steps use an LLM. We use the LLM only where rule-based extraction is insufficient. The first LLM step classifies workflow relevance from downloaded reference-page content. The second LLM step fills supplementary article-level audit flags from the processed text of the articles. The LLM calls use temperature 0 to reduce variation. Repeated execution may still depend on the model and service version used.

The pipeline is largely automated, but it deliberately permits human intervention at two boundary points: obtaining article PDFs at the beginning of the process and obtaining workflow artifacts near the end, before formal result preparation. Both tasks may involve institutional access, protected publisher pages, supplement portals, captchas, and changing web interfaces. The process can therefore benefit from human assistance at the beginning and end, while the intermediate steps execute automatically from recorded evidence.

The implementation was run with Python 3.14.5 and uses a separate `.venv-playwright` virtual environment for browser-dependent steps. Article PDF conversion uses GROBID `0.9.0-crf` through the Docker image `grobid/grobid:0.9.0-crf` [4]. Browser-assisted fetching uses Playwright for Python 1.61.0 with Chromium/Chrome [10]. The two LLM-supported steps call the OpenAI Chat Completions API directly, using the configured model `gpt-4.1-mini` and temperature 0 [11]. OpenAlex is the bibliographic source for article discovery and ranking [13]. The exact commands, script order, prompt files, and intermediate outputs are recorded in `scripts/audit_script_chain.json`.

For reproducibility of the LLM-supported steps, API configuration is separated from prompts and data. The scripts read the API key from `.env` or the process environment as `OPENAI_API_KEY`; this local credentials file is not part of the research data. The model can be overridden with `OPENAI_MODEL`, but the recorded run used the default `gpt-4.1-mini`. Both LLM scripts use the Chat Completions endpoint `https://api.openai.com/v1/chat/completions`, temperature `0.0`, and at most three retries for transient server-side failures. The prompts are stored as JSON configuration files: `data/processed/audit/article_reference_classification_prompt.json` for reference-page classification and `data/processed/audit/article_supplementary_flag_prompt.json` for article-level supplementary flags. The derived JSON outputs record the script name, input file, prompt configuration, model, temperature, selection scope, summary counts, and per-article classifications. This makes the exact prompt and local evidence auditable, while recognizing that future reruns may differ if the hosted model implementation changes.

The following paragraphs describe each step in the ordered script chain. For each step, we identify its purpose, implementation script, principal inputs or outputs, and any point at which manual intervention may be required.

Step 1 collects KNIME-matching article records from the OpenAlex works API with `scripts/collect_openalex_knime_works.py`. The query searches for KNIME in article titles and abstracts and stores the original API response pages, a complete JSONL stream, a compact CSV view, and query metadata. This step preserves the bibliographic source records before any article-level audit decisions are made.

Step 2 builds the bibliometric summaries and the citation-ranked seed table with `scripts/build_openalex_knime_bibliometrics.py`. The script reads the OpenAlex JSONL export and derives processed CSV and JSON files used to select the top-cited audit candidates. The citation-ranked seed table is then used as the article list for downstream download, parsing, URL collection, and audit report construction.

Step 3 attempts to download article PDFs by citation-rank range with `scripts/download_openalex_rank_range_articles.py`. This step is semi-automated because OpenAlex links and open-access URLs do not always expose the full paper, and some papers require lawful institutional access. The script records download attempts and stores retrieved PDFs under `data/original/articles/`; failed or protected records can be completed by manual download without changing the later audit logic.

Step 4 converts the local PDFs into semantic article HTML with `scripts/extract_article_grobid_html.py`. The current parser uses a local GROBID service to produce TEI XML and then renders an HTML reading copy with article metadata, abstract, body sections, paragraphs, and bibliography structure. This replaced earlier plain-text extraction because the downstream steps need stable article identity, paragraphs, and reference context.

Step 5 collects article metadata and article-related URLs with `scripts/collect_article_urls.py`. The script joins the OpenAlex seed table, the canonical PDF registry, the GROBID TEI files, and the generated HTML files. At this stage it does not classify URLs or infer workflow availability; it only records metadata and normalized URLs that can be traced to the article content.

Step 6 fetches the referenced URL pages with `scripts/fetch_article_url_pages.py`. For each unique URL, the script records HTTP status, redirect chain, final URL, response headers, and a stored copy of the response body where available. This creates local evidence for later classification and avoids asking the language model to rely on live web state.

Step 7 attaches the URL-fetch metadata back to the article records with `scripts/attach_url_fetch_metadata.py`. This rule-based step augments each URL entry with fetch status, HTTP code, final URL, redirect count, and the path to any stored page. It is metadata attachment only; it does not interpret whether a reference is a workflow, software page, dataset, article landing page, or documentation.

Step 8 performs a second-stage fetch for reference pages that ordinary HTTP requests cannot access, using `scripts/browser_fetch_403_urls.py`. This step uses a browser when useful, stores rendered pages, and records challenge or blocked states explicitly. The process does not solve captchas or bypass access controls; protected pages are treated as evidence of access friction rather than forced downloads.

Step 9 classifies reference pages for workflow obtainability with `scripts/classify_article_references_with_llm.py`. The language model is given URL metadata and locally cached reference-page excerpts, not the whole article as its primary evidence. The purpose is to decide whether a reference can help a reader obtain a KNIME workflow artifact, whether it only points to software, documentation, data, or a general page, or whether manual checking is needed.

Step 10 classifies supplementary article-level flags with `scripts/classify_article_supplementary_flags_with_llm.py`. This LLM step uses the article HTML, metadata, and reference classifications to fill bounded boolean fields comparable to a manual audit checklist: KNIME use, KNIME version reporting, workflow screenshots or figures, textual workflow or node descriptions, input-data availability, code or scripts, and extension or plugin information. These flags provide article-level context around the reference-page workflow evidence.

Step 11 builds a compact article audit report with `scripts/build_article_audit_report.py`. This rule-based join combines article metadata, the supplementary flag fields, and the workflow-relevant reference classifications into `data/processed/audit/article_audit_report.json`. The report is the central article-level audit artifact used by later workflow-download and table generation steps.

Step 12 attempts automated workflow-artifact downloads from the workflow references in the audit report with `scripts/download_workflow_artifacts.py`. The script uses rule-based HTTP retrieval and, where appropriate, browser-assisted discovery to look for KNIME workflow archives, `workflow.knime` files, or archives containing workflow files. It writes attempt status, local directories, downloaded files, and discovered workflow files to the workflow-reference inventory.

Step 13 is the manual workflow-download checkpoint. The auditor inspects `data/processed/audit/knime_downloadable_workflow_references.json` and tries to download missing workflow artifacts where access is allowed. The inventory JSON is not edited manually at this stage; the manual action is to place obtained files in the article-specific directory under `data/original/workflows/`. This preserves a clear boundary between human retrieval and scripted evidence updating.

Step 14 updates the article report from retrieved workflow evidence with `scripts/update_article_report_from_workflow_inventory.py`. The script scans the workflow inventory and the actual files under `data/original/workflows/` and updates the article audit report when a downloadable KNIME workflow has been obtained. This makes the report reflect the local evidence rather than only the earlier URL classification.

Step 15 generates the article-audit summary table source with `scripts/build_article_audit_tables.py`. The script reads the compact article audit report and writes the CSV source for the publication-level flag summary. The resulting data are therefore derived from the final structured report rather than maintained by hand.

Step 16 generates the KNIME-use workflow-reporting table source with `scripts/build_knime_use_workflow_reporting_table.py`. This step combines the article-level audit report with the workflow-reference inventory, so that publication-level workflow-reporting signals and actual obtained workflow evidence are kept separate but can be summarized consistently.

The resulting evidence pipeline is reproducible at the level of inputs, scripts, configuration files, cached pages, LLM prompts, and derived JSON reports, while still acknowledging that publication and workflow retrieval are not fully automatable. Manual actions are restricted to acquiring protected article PDFs and workflow artifacts where access is lawful; the subsequent interpretation, report updating, and table construction are scripted from recorded evidence.

All project Python scripts are intended to be run with Python 3.11 or newer. The current scripts use only the Python standard library. No third-party Python packages are required for the OpenAlex collection and rule-based audit steps. Browser-assisted, GROBID, and LLM-supported steps use the separately recorded tools and services described above.

4 Data

4.1 Bibliometric Evidence

The OpenAlex bibliometric data are organized into original API responses and processed analysis tables. The original data are stored under `data/original/openalex/`. They include raw response pages, a complete JSONL export, a compact CSV export, and query metadata. These files preserve the article-level records used to measure the size, time trend, venues, fields, and accessibility of the KNIME-matching literature.

Derived bibliometric tables are stored under `data/processed/openalex/`. Their roles in the analysis are:

- `data/processed/openalex/openalex_knime_articles.csv`: compact article-level metadata used as the main processed bibliography table.
- `data/processed/openalex/openalex_knime_articles_by_year.csv`: annual publication counts used to describe the time trend of KNIME-matching literature.
- `data/processed/openalex/openalex_knime_top_venues.csv`: source-level aggregation used to identify journals, proceedings, repositories, and platforms where the matching records appear.
- `data/processed/openalex/openalex_knime_top_domains.csv`: OpenAlex domain aggregation used to describe the broad disciplinary distribution.
- `data/processed/openalex/openalex_knime_top_fields.csv`: OpenAlex field aggregation used to describe the finer disciplinary distribution.
- `data/processed/openalex/openalex_knime_most_cited.csv`: citation-ranked subset used to select influential records for later manual assessment.
- `data/processed/openalex/openalex_knime_likely_workflow_platform.csv`: heuristic subset used to prioritize papers likely to use KNIME as a workflow platform.

The processed OpenAlex directory also contains additional summary files:

- `data/processed/openalex/openalex_knime_open_access_availability.json`
- `data/processed/openalex/openalex_knime_role_classification_summary.json`

These files summarize artifact-access signals and role-classification counts.

4.2 Top-Cited Article Assessment

The top-cited article dataset records the structured assessment of the 100 most-cited KNIME-matching OpenAlex records. The citation-ranked source list is `data/processed/openalex/openalex_knime_most_cited.csv`, and the local article registry is stored under `data/original/articles/`. Local PDFs, where available, are preserved in that directory, while download-attempt logs are stored under `data/processed/audit/logs/`.

Article text and references are represented by GROBID outputs. Semantic HTML files are stored under `data/processed/articles/`, and TEI XML files are stored under `data/processed/articles/grobid_tei/`. These outputs provide article text, section structure, bibliography context, and machine-readable links for the later URL and LLM steps.

The URL extraction step writes `data/processed/audit/article_url_collection.json`. For each article, this file stores rank, canonical identifiers, PDF path, TEI path, HTML path, OpenAlex metadata, and article-related URLs extracted from the GROBID outputs. The fetched reference-page cache is stored under `data/processed/audit/pages/`; browser-rendered fetches for selected blocked pages are stored under `data/processed/audit/browser_pages/`. These caches include HTTP status, redirects, final URLs, headers, stored body paths, and fetch errors or challenge pages where applicable.

The LLM-derived article-audit data are split into two files. `data/processed/audit/article_reference_llm_classifications.json` classifies cached article references by workflow obtainability and records the page evidence used for those classifications. `data/processed/audit/article_supplementary_llm_flags.json` records article-level flags for KNIME use, version reporting, workflow figures, workflow text descriptions, data availability, code availability, and extension information.

The compact report used for the article-level tables is `data/processed/audit/article_audit_report.json`. It merges article metadata, supplementary flags, and workflow-relevant reference classifications into one record per article. Workflow-download attempts and local workflow paths are recorded in `data/processed/audit/knime_downloadable_workflow_references.json`; manual KNIME-opening screenshots are stored under each workflow directory's `opened/` subdirectory. Table sources are generated under `article_bibliometric/tables/` from these JSON files rather than edited manually.

5 Results

5.1 Bibliometric Evidence

The OpenAlex title-and-abstract query returned 963 article-type records. This establishes that KNIME is visible in a sizeable scientific literature and that workflow preservation is relevant beyond platform-maintenance research. The annual series indicates that KNIME-matching literature appears in OpenAlex from the mid-2000s and grows into a sustained publication stream after 2015 (Fig. 1). The highest annual count in the current export is 104 records in 2023. Counts for 2026 are incomplete because the query was collected on June 29, 2026.

The domain distribution shows that the KNIME-matching literature is not confined to one disciplinary area (Table 1). OpenAlex assigns just over half of the records to Physical Sciences, but the remaining records are spread across Social Sciences, Life Sciences, and Health Sciences. This matters for the study because workflow reproducibility risk is not only a software-engineering concern. If KNIME workflows are used across computational chemistry, biomedical analysis, education, business analytics, and related areas, then missing workflow artifacts and undocumented dependencies can affect published computational evidence in several research communities.

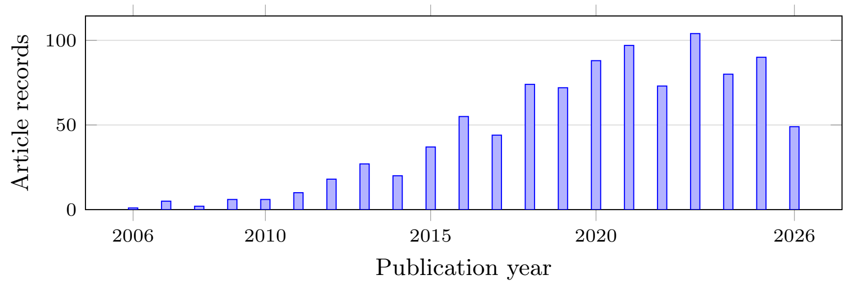

Fig. 1. OpenAlex article records matching `KNIME` in title or abstract by publication year.

Table 1. OpenAlex domains for KNIME title-and-abstract matches.

Domain	Articles	Share
Physical Sciences	505	52.44%
Social Sciences	187	19.42%
Life Sciences	149	15.47%
Health Sciences	122	12.67%

The ten largest OpenAlex fields are led by Computer Science (350), Biochemistry, Genetics and Molecular Biology (115), Medicine (92), Engineering (71), and Social Sciences (66). These field counts show why a paper about KNIME reproducibility cannot focus only on the KNIME platform literature. The central reproducibility question is how scientific studies that use KNIME as a tool report the executable workflow, data, versions, and extensions needed for later reproduction.

Open-access metadata indicate 619 open-access records, 625 records with some full-text availability, and 421 records with a PDF signal. These signals are important because later workflow audits require access to paper text, supplements, repositories, and sometimes PDF-only workflow descriptions. The counts are only availability indicators. They do not by themselves establish that a workflow artifact, KNIME version, input data, or extension list is available.

The heuristic role classification separates likely papers about KNIME or KNIME extensions from papers that likely use KNIME as an analysis tool. The current rules classify 156 records as about KNIME or a KNIME extension, 314 as using KNIME as a tool, 28 as about or mentioning KNIME in the title, and 465 as ambiguous OpenAlex matches. The same heuristic marks 410 records as likely using KNIME as a workflow platform. These labels are useful for triage: they identify a substantial candidate set where workflow reporting can matter, but final case-study claims require manual full-text and artifact assessment.

5.2 Top-Cited Article Assessment

Of the 100 citation-ranked KNIME-matching records, 37 article PDFs were downloaded by the retrieval script and 42 were obtained manually, giving 79 local PDFs in total. The top-cited audit therefore shows that article access itself is already a limiting condition for reproducibility assessment. This means that the automated middle of the pipeline can evaluate most, but not all, top-cited records from local article text and extracted references. The remaining records stay in the denominator, because excluding them would overstate the assessability of the literature.

The URL and reference-page stages add a more precise view of workflow availability than article text alone. The compact report contains workflow-relevant linked resources for 22 article records, with 68 linked resources retained after filtering generic article pages, datasets, software documentation, and malformed or unrelated URLs. This result is important methodologically: the clearest evidence for whether a reader can obtain a workflow can appear on the referenced page itself rather than in the sentence that cites the URL.

The LLM reference-classification step reduced the URL set to resources that could plausibly support workflow retrieval and assigned bounded categories for downloaded artifacts, workflow landing pages, direct workflow links, and resources requiring manual inspection. The broader workflow inventory retains 31 candidate workflow-reference records because it preserves retrieval attempts even when the compact article-level flag is stricter. This separation allows the audit to distinguish evidence that an article or reference suggests a workflow is available from evidence that a workflow artifact was obtained locally.

The supplementary LLM flag step provides the publication-level reporting context for those workflow links. Seventy-five records use KNIME as a workflow, tool, interface, or implementation context. Of the full 100-record registry, 68 records describe workflow steps or nodes in text, and 49 show workflow screenshots or figures. In contrast, 18 report a KNIME version, 22 report a downloadable or linked workflow, 23 report extension dependencies, and four report an extension installation source. Visual or narrative workflow presentation is therefore more common than the executable and environment context needed for later reproduction.

Restricting the reporting signals to the 75 KNIME-use cases changes two of the all-record counts: 48 of the KNIME-use articles show workflow screenshots or figures, and 22 report extension or plugin dependencies. The corresponding all-record counts are 49 and 23 because each signal also occurs once in a comparative paper that is not classified as a KNIME-use case. The remaining signals in Table 3 are fully contained within the 75 KNIME-use cases.

The workflow-download stage converts reported or linked workflow evidence into local artifact evidence. Eighteen article records yielded workflow files, archives, or workflow directories. Four were obtained automatically and fourteen required manual download. This confirms the semi-automated character of the boundary step: scripted retrieval can find some artifacts, but protected pages, stale links, repository layouts, and supplement portals still require human inspection.

Table 2 summarizes the statistics-ready fields in the compact article audit report. Data and code signals are recorded separately from workflow-artifact evidence. We do not verify the availability of reported data or code in this table; we record whether the article and linked evidence report those resources.

Table 2: Flag summary for the top-cited article audit. Shares use all 100 records in the registry as the denominator.

Audit category or flag	Count	Share of 100	Interpretation
Audit records	100	100.0%	Most-cited KNIME-matching article records selected from OpenAlex
Records with local PDF	79	79.0%	Source PDF recorded in the compact audit report
Uses KNIME	75	75.0%	KNIME used as a workflow, tool, interface, or implementation context
Workflow or nodes described in text	68	68.0%	Named workflow steps, nodes, modules, or components recorded
Workflow screenshots or figures	49	49.0%	Workflow shown visually in article or supplement
Reports KNIME version	18	18.0%	Specific KNIME version value found
Reports downloadable workflows	22	22.0%	Executable workflow files provided or linked
Reports extension/plugin dependencies	23	23.0%	KNIME extension or node-library dependency reported
Reports extension installation source	4	4.0%	Update site, project URL, or installation source reported
Provides code or scripts	20	20.0%	Code repository, scripts, or software code reported
Reports input-data availability	66	66.0%	Data availability stated, with or without direct URL
Direct input-data resource	20	20.0%	Direct data URL or resource pointer recorded

Table 3: Workflow-reporting and retrieval signals among the 75 assessed KNIME-use cases.

Audit flag	Count	Share of KNIME-use cases
Uses KNIME	75	100.0%
Workflow or nodes described in text	68	90.7%
Workflow screenshots or figures	48	64.0%
Reports KNIME version	18	24.0%
Reports extension/plugin dependencies	22	29.3%
Reports extension installation source	4	5.3%
Reports downloadable workflows	22	29.3%
Articles with successfully downloaded workflows	18	24.0%

For workflow retrieval, the authoritative data source is the workflow inventory derived from the compact audit report and the local workflow directories. Table 4 summarizes the recorded artifact-retrieval outcomes used for manuscript inspection. In this table, the *Success* column marks whether a KNIME workflow artifact was actually obtained: **X** means that a workflow file, archive, or workflow directory was retrieved, while **-** means that no KNIME workflow artifact was obtained or confirmed. Failed or incomplete retrievals are recorded separately from article-level claims that a workflow was reported or linked.

Table 5 then reports the manual opening and execution checks in the current KNIME environment for the downloaded article records. Here, the *Executed* column has a stricter meaning: **X** means that at least one workflow for the article opened and executed successfully in the tested environment, while **-** means that the workflow opened but did not have a confirmed successful end-to-end run, or failed during execution. Five article records had at least one workflow execute successfully in the recorded manual checks: PAINS, Webinar Pricing Analytics, ImageJ ecosystem integration, high-content organelle trafficking, and GediNET [16, 18, 2, 12, 14]. Other opened workflows did not have a confirmed successful end-to-end run in this manual check. This does not show that those workflows are unreproducible: they may require additional configuration, dependencies, or repair before a successful run. The screenshots from the opening checks are archived under each workflow directory's `opened/` subdirectory in `data/original/workflows/`.

Table 4: Workflow retrieval results for article records with attempted or recorded artifact retrieval outcomes.

#	Article record	Success	Retrieval note
1	PAINS workflows, DOI `10.1002/minf.201100076`	**X**	Automated myExperiment page and download requests returned HTTP 403 HTML, but we manually obtained both RDKit and Indigo workflow ZIP archives. Both contain `workflow.knime` files.
2	Chemical data curation workflow, DOI `10.1186/s13321-018-0315-6`	**X**	GitHub master archive was downloaded after retry and passed ZIP integrity checking. KNIME workflow files were found in the archive.
3	ASD genes prediction workflow, DOI `10.1371/journal.pone.0208626`	**X**	GitHub archive passed ZIP integrity checking. An inner KNIME workflow archive was extracted to a workflow directory.
4	Bitterness/sweetness classifier workflow, DOI `10.3389/fchem.2018.00093`	**-**	The original project page redirected to VirtualTaste, and the redirected page returned HTTP 404. No workflow archive was retrieved.
5	Medicinal chemistry filters workflow, DOI `10.3390/encyclopedia3020035`	**X**	GitLab archive passed ZIP integrity checking and contained KNIME workflow files. One KNIME Hub URL returned a page, while an older generic Hub URL returned 404 JSON.
6	MS-ready structures repository, DOI `10.1186/s13321-018-0299-2`	**-**	The GitHub archive passed ZIP integrity checking, but the extracted repository contained only `LICENSE` and `README.md`. No `.knwf` or `workflow.knime` file was found.
7	KNIME-OMICS reference, DOI `10.1016/j.jprot.2016.08.002`	**-**	The referenced GitHub organization page returned HTTP 404. No workflow archive was retrieved.
8	QSAR tools workflow, DOI `10.1080/07391102.2018.1456975`	**-**	The referenced `teqip.jdvu.ac.in/QSAR_Tools/` URL failed DNS resolution. No workflow archive was retrieved.

#	Article record	Success	Retrieval note
9	Webinar Pricing Analytics workflow, DOI 10.1016/j.jbusres.2021.08.036	**X**	We manually downloaded the KNIME workflow file and added local KNIME opening screenshots.
10	Verhaar scheme workflow, DOI 10.1016/j.chemosphere.2015.06.009	-	The article reports a supplementary KNIME workflow. Standard Elsevier supplement URLs resolved as Excel files, but no direct KNIME workflow archive URL was confirmed.
11	Spontaneous tail coiling workflow, DOI 10.1016/j.ntt.2020.106918	**X**	We manually downloaded the KNIME workflow file and added local KNIME opening screenshots.
12	Caco-2 permeability prediction workflow, DOI 10.3390/pharmaceutics14101998	**X**	We manually downloaded the KNIME workflow file and added local KNIME opening screenshots.
13	KniMet workflow, DOI 10.1007/s11306-018-1349-5	**X**	The Zenodo DOI resolved to record 1196407. `KniMet-master.zip` was downloaded and contains `KniMet.knwf`.
14	NGS sample workflow, DOI 10.1093/bioinformatics/btr478	**X**	We manually downloaded the myExperiment archive after automated requests were blocked by HTTP 403. The ZIP contains a KNIME workflow directory with saved data.
15	ImageJ ecosystem workflows, DOI 10.3389/fcomp.2020.00008	**X**	Six KNIME Hub artifact metadata endpoints returned signed URLs and the corresponding `.knwf` files were downloaded. One recorded reference appears stale or unavailable.
16	Multi-template matching repository, DOI 10.1186/s12859-020-3363-7	-	The GitHub archive was downloaded, but it contained documentation/images only and no `.knwf`, `workflow.knime`, or KNIME workflow archive. A guessed NodePit URL returned HTTP 404.
17	GediNET workflows, DOI 10.1038/s41598-022-24421-0	**X**	GitHub master archive was downloaded and contains two `.knwf` files. The recorded KNIME Hub short link could not be retrieved.
18	Nuclear receptor ligand workflow, DOI 10.1002/minf.201400188	-	The article reports supplementary workflow material, but Wiley article and guessed supplementary download endpoints returned Cloudflare challenge pages. Crossref metadata did not expose a supplementary workflow URL.
19	High-content organelle trafficking workflow, DOI 10.1038/sdata.2018.241	**X**	The figshare collection DOI resolved through the figshare API. The default KNIME workflow file was downloaded as `KNIME workflow.zip`.
20	Knime4Bio NOTCH2 workflow, DOI 10.1093/bioinformatics/btr554	**X**	The article reports that the workflow was posted on myExperiment. The local manual artifact is `The_NOTCH2_workflow-v1.zip`, which contains a KNIME workflow directory.

Table 5: Manual opening and execution results for downloaded KNIME workflow artifacts.

#	Article record	Executed	Manual KNIME result
1	PAINS workflows, DOI 10.1002/minf.201100076	**X**	RDKit and Indigo workflow ZIPs were tested. At article-record level, at least one workflow opened and executed successfully. The separate Indigo workflow opened but was blocked by a missing Indigo KNIME Integration node.
2	Chemical data curation workflow, DOI 10.1186/s13321-018-0315-6	-	The CompTox rebuilt workflow file opened, but execution failed inside a metanode during the manual test.

#	Article record	Executed	Manual KNIME result
3	ASD genes prediction workflow, DOI `10.1371/journal.pone.0208626`	-	The extracted ASD genes prediction workflow opened, but execution failed in an R node because required R packages and functions were unavailable in the test environment.
4	Medicinal chemistry filters workflow, DOI `10.3390/encyclopedia3020035`	-	The medicinal chemistry filters workflow file opened, but a successful end-to-end execution was not established in the available notes.
5	Webinar Pricing Analytics workflow, DOI `10.1016/j.jbusres.2021.08.036`	X	The Webinar Pricing Analytics workflow file opened and executed successfully in the current local KNIME environment.
6	Spontaneous tail coiling workflow, DOI `10.1016/j.ntt.2020.106918`	-	The spontaneous tail coiling measurement workflow file opened in the current local KNIME environment, but a successful end-to-end execution was not established in the available notes.
7	Caco-2 permeability prediction workflow, DOI `10.3390/pharmaceutics14101998`	-	The manual Caco-2 prediction `.knwf` file opened in the current local KNIME environment, but a successful end-to-end execution was not established in the available notes.
8	KniMet workflow, DOI `10.1007/s11306-018-1349-5`	-	The KniMet workflow from the Zenodo archive opened in the current local KNIME environment, but successful execution was not established in the available notes.
9	NGS sample workflow, DOI `10.1093/bioinformatics/btr478`	-	The NGS sample workflow archive with saved data opened in the current local KNIME environment, but successful execution was not established in the available notes.
10	ImageJ ecosystem workflows, DOI `10.3389/fcomp.2020.00008`	X	Six downloaded KNIME Hub `.knwf` files opened and executed normally in our test environment after installing additional required KNIME/ImageJ-related plugins.
11	GediNET workflows, DOI `10.1038/s41598-022-24421-0`	X	The GediNET GitHub archive opened in the current local KNIME environment and the workflow was run successfully in the manual check.
12	High-content organelle trafficking workflow, DOI `10.1038/sdata.2018.241`	X	`KNIME workflow.zip` opened and executed normally in our test environment after installing additional required plugins.
13	Knime4Bio NOTCH2 workflow, DOI `10.1093/bioinformatics/btr554`	-	`The_NOTCH2_workflow-v1.zip` opened in the current local KNIME environment, but a successful end-to-end execution was not established in the available notes.

Taken together, the results show three stages of evidence. First, KNIME is visible in a broad and sustained scientific literature. Second, reporting signals are uneven: 68 records describe workflow steps or nodes in text, 49 show workflow screenshots or figures, 22 report downloadable or linked workflows, and 18 yielded workflow artifacts or directories. Third, workflows from all 18 records with obtained artifacts opened in the current local KNIME environment, but only five article records had at least one workflow execute successfully in the recorded checks. These findings do not show that KNIME workflows generally fail. They identify a narrower reproducibility risk: workflow images and textual descriptions do not provide the executable and environmental evidence needed to test, diagnose, and reproduce the reported analysis.

6 Discussion

This study contributes a semi-automated evidence pipeline and a focused empirical account of how KNIME workflows are reported and preserved in scientific publications. Its scope is intentionally narrower than a full reproducibility benchmark: we do not estimate the failure rate of all published KNIME workflows. Instead, the pipeline links bibliometric records, publication-level reporting signals, referenced web resources, retrieved artifacts, and manual opening and execution outcomes. The resulting contrast is clear: 68 records contain textual workflow descriptions, 22 report downloadable or linked workflows, 18 yielded artifacts or directories, and five had a confirmed successful execution.

The results suggest three practical implications. First, a published workflow image is not an executable artifact. Several highly cited papers provide figures or textual descriptions of connected KNIME nodes, but this is not enough to recover node settings, extension versions, or input paths. Second, reporting a KNIME version is useful but incomplete. Workflows may also depend on third-party extension update sites and node libraries, as shown by the PAINS case where the RDKit workflow could be updated and executed while the Indigo workflow was blocked by a missing extension. Third, a reported or linked workflow should not be counted as an obtained artifact without checking the target resource. The difference between 22 records reporting downloadable or linked workflows and 18 records yielding workflow artifacts or directories demonstrates the importance of preserving retrieval evidence separately from publication-level reporting.

The study has several limitations. The OpenAlex query identifies records whose titles or abstracts match `KNIME`; it is not a complete census of every study that used the platform. The top-cited audit is designed to examine influential records and is not a random sample. Local full text was available for 79 of the 100 records, and inaccessible records remain in the denominator. The audit records whether articles report data and code availability but does not verify every such resource. Finally, the opening and execution checks reflect the recorded local environment and available configuration effort. They support case-level outcomes, not a general estimate of workflow executability.

The next development of this study should move from paper-level evidence to workflow-level evidence. One direction is to create a larger corpus of public KNIME workflows from supplements, myExperiment, KNIME Hub, GitHub, and other repositories, then test whether they open and execute across documented KNIME versions and extension configurations. A second direction is to preserve reproducibility environments for selected case studies by recording operating system, KNIME version, installed extensions, update sites, input data, import warnings, execution errors, and output differences.

Tools and services outside the present evidence base could provide a further layer of usage evidence. `knime2py` could help identify KNIME workflows that users try to translate or operationalize outside KNIME [7], while `k2pweb.org` could provide anonymized, aggregate evidence about uploaded or executed workflows [6]. These sources could help answer a question that the current study leaves open: which access, dependency, configuration, and execution barriers researchers encounter when they attempt to reuse KNIME workflows in practice.

7 Conclusion

The bibliometric and audit evidence distinguishes the visibility of KNIME in scientific literature from the preservation of executable KNIME analyses. Of 100 top-cited KNIME-matching records, 75 were classified as KNIME-use cases, 22 reported downloadable or linked workflows, 18 yielded workflow artifacts or workflow directories, and five had at least one workflow execute successfully in the recorded checks. These results do not imply that KNIME workflows are generally unreproducible. They show that long-term assessment depends on publishing the executable workflow together with version, extension, data, and execution context, and on recording whether linked artifacts remain retrievable and runnable.

References

1. Berthold, M.R., Cebron, N., Dill, F., Gabriel, T.R., Kötter, T., Meinl, T., Ohl, P., Sieb, C., Thiel, K., Wiswedel, B.: KNIME: The Konstanz information miner. In: Data Analysis, Machine Learning and Applications, pp. 319–326. Springer (2008). https://doi.org/10.1007/978-3-540-78246-9_38
2. Dietz, C., Rueden, C.T., Helfrich, S., Dobson, E.T.A., Horn, M., Eglinger, J., Evans, E.L., McLean, D.T., Novitskaya, T., Ricke, W.A., Sherer, N.M., Zijlstra, A., Berthold, M.R., Eliceiri, K.W.: Integration of the ImageJ ecosystem in KNIME analytics platform. Frontiers in Computer Science **2**, 8 (2020). https://doi.org/10.3389/fcomp.2020.00008
3. Fu, X., Liu, L., Guan, W.W., Kalra, Y., Bao, S., Kötter, T., Sturm, K.: Advancing replicable and reproducible GIScience: an approach with KNIME. Cartography and Geographic Information Science **53**(3), 270–290 (2025). https://doi.org/10.1080/15230406.2024.2446556
4. GROBID contributors: GROBID: Generation of bibliographic data. `https://github.com/kermitt2/grobid` (2026), version 0.9.0-crf Docker image; accessed July 12, 2026
5. Hall, M., Frank, E., Holmes, G., Pfahringer, B., Reutemann, P., Witten, I.H.: The WEKA data mining software: An update. ACM SIGKDD Explorations Newsletter **11**(1), 10–18 (2009). https://doi.org/10.1145/1656274.1656278
6. Kaplan, V.: k2p-web: KNIME to Python/Jupyter converter. `https://k2pweb.org/` (2026), accessed July 1, 2026
7. Kaplan, V.: knime2py: KNIME to Python workbook. `https://github.com/vitalii-kaplan/knime2py` (2026), accessed July 1, 2026
8. Kaplan, V.: reproducibility-risks-article: Replication repository for KNIME workflow reproducibility-risk study. `https://github.com/vitalii-kaplan/reproducibility-risks-article` (2026), accessed July 1, 2026
9. Kurtzer, G.M., Sochat, V., Bauer, M.W.: Singularity: Scientific containers for mobility of compute. PLOS ONE **12**(5), e0177459 (2017). https://doi.org/10.1371/journal.pone.0177459

10. Microsoft: Playwright for Python. https://playwright.dev/python/ (2026), version 1.61.0; accessed July 12, 2026
11. OpenAI: OpenAI API Documentation: Chat completions. https://platform.openai.com/docs/api-reference/chat (2026), accessed July 12, 2026
12. Pal, A., Glaß, H., Naumann, M., Kreiter, N., Japtok, J., Sczech, R., Hermann, A.: High content organelle trafficking enables disease state profiling as powerful tool for disease modelling. Scientific Data **5**(1), 180241 (2018). https://doi.org/10.1038/sdata.2018.241
13. Priem, J., Piwowar, H., Orr, R.: OpenAlex: A fully-open index of scholarly works, authors, venues, institutions, and concepts. arXiv preprint arXiv:2205.01833 (2022). https://doi.org/10.48550/arXiv.2205.01833, https://arxiv.org/abs/2205.01833
14. Qumsiyeh, E., Showe, L.C., Yousef, M.: GediNET for discovering gene associations across diseases using knowledge based machine learning approach. Scientific Reports **12**(1), 19955 (2022). https://doi.org/10.1038/s41598-022-24421-0
15. Samuel, S., Mietchen, D.: Computational reproducibility of Jupyter notebooks from biomedical publications. GigaScience **13** (2024). https://doi.org/10.1093/gigascience/giad113
16. Saubern, S., Guha, R., Baell, J.B.: KNIME workflow to assess PAINS filters in SMARTS format: Comparison of RDKit and Indigo cheminformatics libraries. Molecular Informatics **30**(10), 847–850 (2011). https://doi.org/10.1002/minf.201100076
17. Tahir, A., Rasheed, S., Dietrich, J., Hashemi, N., Zhang, L.: Test flakiness' causes, detection, impact and responses: A multivocal review. Journal of Systems and Software **206**, 111837 (2023). https://doi.org/10.1016/j.jss.2023.111837
18. Villarroel Ordenes, F., Silipo, R.: Machine learning for marketing on the KNIME hub: The development of a live repository for marketing applications. Journal of Business Research **137**, 393–410 (2021). https://doi.org/10.1016/j.jbusres.2021.08.036
19. Walker, R., Slingsby, A., Dykes, J., Xu, K., Wood, J., Nguyen, P.H., Stephens, D., Wong, B.L.W., Zheng, Y.: An extensible framework for provenance in human terrain visual analytics. IEEE Transactions on Visualization and Computer Graphics **19**(12), 2139–2148 (2013). https://doi.org/10.1109/TVCG.2013.132